AWT List Component – Multiple Selection for Bulk SKU Updates

CO – 4 Assessment Tool - 1

Name : Tamil Elakiya S

Reg No : 192524443

Aim

To modify a legacy Java AWT inventory GUI so that warehouse staff can select multiple SKUs simultaneously using the List component and retrieve all selected items for a bulk price update.

Problem Statement

A legacy inventory application uses the Java AWT List component to display SKUs. The existing application handles only one selected SKU at a time.

The requirement is to allow warehouse staff to select multiple SKUs simultaneously and perform a bulk price update.

The AWT List component supports this through **multiple-selection mode**. The application must also change its event-handling code so that it retrieves all selected items instead of retrieving only one selected item.

AWT List Component

The Java AWT List component is a GUI component that displays a list of text items. It can be configured to allow either single selection or multiple selection.

The constructor is:

List(int rows, boolean multipleMode)

The second parameter determines whether multiple items can be selected.

For multiple selection:

List skuList = new List(8, true);

Method to Enable Multiple Selection

The AWT List component also provides the following method:

setMultipleMode(boolean b)

Therefore, an existing List can be changed to multiple-selection mode using:

skuList.setMultipleMode(true);

The corresponding method to check the mode is:

isMultipleMode()

Thus, the two important approaches are:

List skuList = new List(8, true);

or:

List skuList = new List(8, false);

skuList.setMultipleMode(true);

Single-Selection Legacy Implementation

The following example represents the existing legacy approach:

```
import java.awt.*;
import java.awt.event.*;

public class InventoryLegacy extends Frame
        implements ActionListener {

    List skuList;
    Button updateButton;
    TextField priceField;
    Label result;

    InventoryLegacy() {

        setTitle("Inventory Price Update");
        setSize(400, 300);
        setLayout(new FlowLayout());

        skuList = new List(8, false);

        skuList.add("SKU-101");
```

```
        skuList.add("SKU-102");
        skuList.add("SKU-103");
        skuList.add("SKU-104");
        skuList.add("SKU-105");

        priceField = new TextField(10);
        updateButton = new Button("Update Price");
        result = new Label("Select an SKU.");

        updateButton.addActionListener(this);

        add(new Label("Select SKU:"));
        add(skuList);
        add(new Label("New Price:"));
        add(priceField);
        add(updateButton);
        add(result);

        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {

        String selectedSKU = skuList.getSelectedItem();

        if (selectedSKU != null) {
            result.setText(
                "Updating price for: " + selectedSKU
            );
        } else {
            result.setText("Please select an SKU.");
        }
    }

    public static void main(String[] args) {
        new InventoryLegacy();
    }
}
```

Complete Modified Java Program

```
import java.awt.*;
import java.awt.event.*;

public class InventoryBulkUpdate extends Frame
        implements ActionListener {

    List skuList;
    Button updateButton;
    TextField priceField;
    TextArea result;

    InventoryBulkUpdate() {

        setTitle("Inventory Bulk Price Update");
        setSize(500, 400);
        setLayout(new FlowLayout());

        // Enable multiple selection
        skuList = new List(8, true);

        skuList.add("SKU-101");
        skuList.add("SKU-102");
        skuList.add("SKU-103");
        skuList.add("SKU-104");
        skuList.add("SKU-105");

        priceField = new TextField(10);

        updateButton = new Button("Update Price");

        result = new TextArea(10, 40);
        result.setEditable(false);

        updateButton.addActionListener(this);

        add(new Label("Select SKUs:"));
        add(skuList);

        add(new Label("New Price:"));
        add(priceField);

        add(updateButton);
```

```
    add(result);

    addWindowListener(new WindowAdapter() {
        public void windowClosing(WindowEvent e) {
            dispose();
        }
    });

    setVisible(true);
}

public void actionPerformed(ActionEvent e) {

    // Retrieve ALL selected SKUs
    String[] selectedSKUs =
            skuList.getSelectedItems();

    String newPrice = priceField.getText();

    if (selectedSKUs.length == 0) {

        result.setText(
            "Please select at least one SKU."
        );

        return;
    }

    if (newPrice.isEmpty()) {

        result.setText(
            "Please enter a new price."
        );

        return;
    }

    result.setText(
        "Bulk price update initiated:\n\n"
    );

    for (String sku : selectedSKUs) {
```

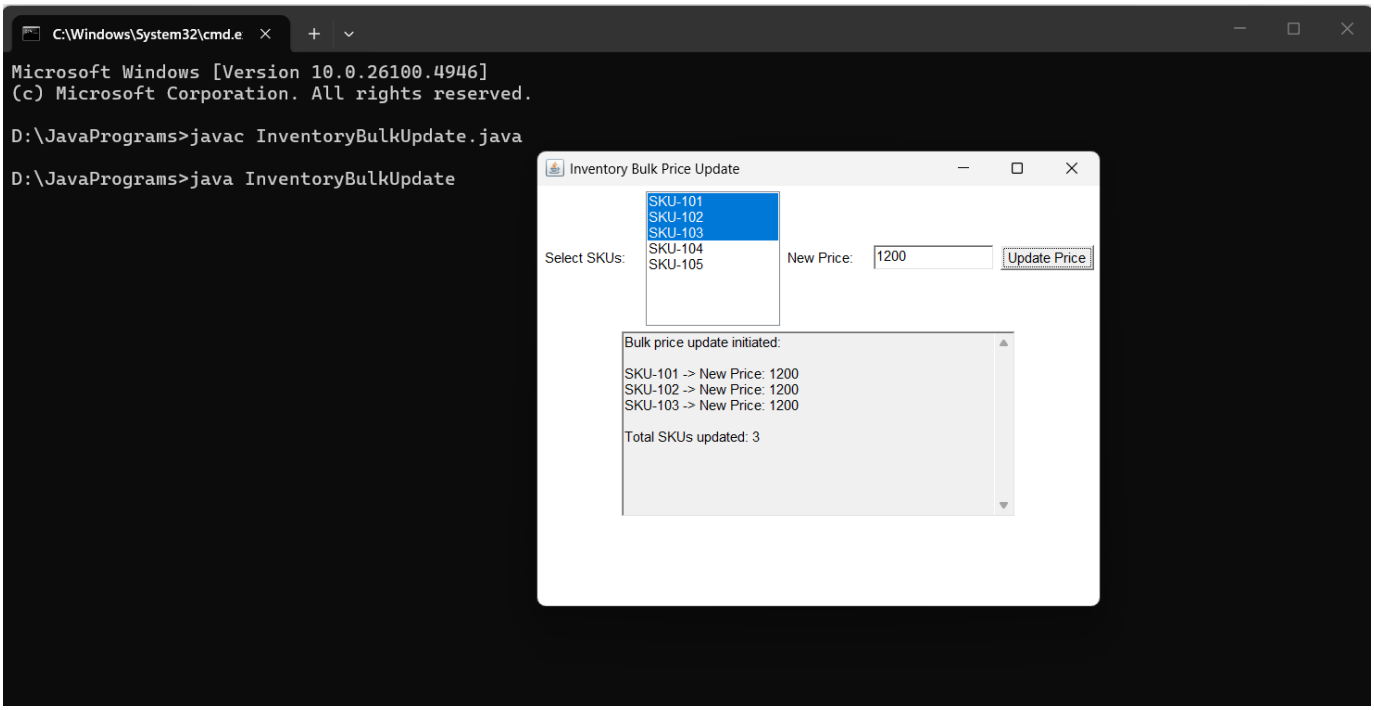
```
            result.append(
                sku + " -> New Price: "
                + newPrice + "\n"
            );
        }

        result.append(
            "\nTotal SKUs updated: "
            + selectedSKUs.length
        );
    }

    public static void main(String[] args) {

        new InventoryBulkUpdate();
    }
}
```

Output :



Conclusion :

The Java AWT List component supports multiple selection by enabling its **multiple-selection mode**. This can be done while creating the component using:

List skuList = new List(8, true);

or after creation using:

skuList.setMultipleMode(true);

The legacy event-handling code uses:

getSelectedItem()

which returns only one selected item. For bulk operations, it must be replaced with:

getSelectedItems()

which returns a String[] containing all selected items. The application can then iterate through this array using a loop and apply the price update to every selected SKU.

Therefore, enabling multiple-selection mode and changing the event-handling logic from getSelectedItem() to getSelectedItems() converts the legacy single-SKU inventory operation into an efficient bulk price-update operation.